SMATCH - Smart Async Task Chaining

Salesforce Asynchronous Framework Documentation

Version: February 2026 – Repository: 20260215

Table of Contents

1. Introduction
2. Implementation Choices
3. Administrator Manual
4. Developer Manual
5. Appendix

I. Introduction

I.A. Vision

SMATCH (SMart Async Task CHaining) is an asynchronous framework for Salesforce that enables orchestrating, executing, and monitoring background tasks reliably and efficiently. The goal is to provide a reusable foundation for all deferred execution needs (emails, integrations, heavy processing, maintenance) without proliferating ad hoc mechanisms specific to each project.

I.B. Problem Addressed

Salesforce organizations often need to:

- Execute heavy processing without blocking users
- Schedule tasks at specific or recurring times
- Handle large volumes while respecting platform limits (CPU, queries, DML)
- Track execution status and diagnose errors
- Adapt behavior without redeploying code

Without a centralized framework, each team develops isolated solutions (Queueable, Schedulable, Batch) that are duplicated, hard to evolve, and often lack visibility.

I.C. Framework Objectives

I.C.1 Simplicity for the Developer

- **Single interface**: implementing IAsyncTask with a run(SF_Async__c) method is enough to create an executable task
- **Task registration** via SF_Async__c records (timestamp, order, class, target)
- **No manual scheduling**: the Worker handles queuing and execution

I.C.2 Performance and Robustness

- **Smart batching**: batch size calculated dynamically from historical metrics (average CPU per task)
- **Per-task and per-class limits**: Tasks_Per_Worker_Limit__c, Max_Tasks_Per_Run_Learned__c (callout/DML errors)
- **Limit compliance**: early stop when CPU or DML approaches the limit, configurable safety margin
- **Grouping by class**, FOR UPDATE to avoid duplicate processing

I.C.3 Observability

- Detailed execution logs (CPU, SOQL, Heap) in SF_Async_Log__c
- Performance statistics per class (SF_Async_Performance__c)
- Events and alerts on error (Platform Event Async_WakeUp__e)
- Invocable diagnostic displaying the full framework state

I.C.4 Configuration Without Deployment

- Parameters centralized in Custom Metadata (Async_Framework_Configuration__mdt)
- Runtime overrides via Custom Settings (Async_Runtime_Config__c)
- Adjustment of thresholds, frequencies, and batch sizes without Apex deployment
- Flow interface to control the framework

I.C.5 Service Continuity

- **Watchdog** (AsyncScheduler) to detect absence of active Workers and trigger wake-up
- **Automatic rescheduling** of the Worker according to configured frequency
- **Automatic cleanup** of old records to limit database footprint
- **Force Start** to manually restart the system

I.D. Target Audience

Role	Usage
Salesforce Developers	Fast integration of async tasks via IAsyncTask interface
Architects	Standardization of async processing across the org
Administrators	Monitoring and configuration via Flow without development

Support Teams	Diagnostic and restart via Smatch interface

I.E. Expected Benefits

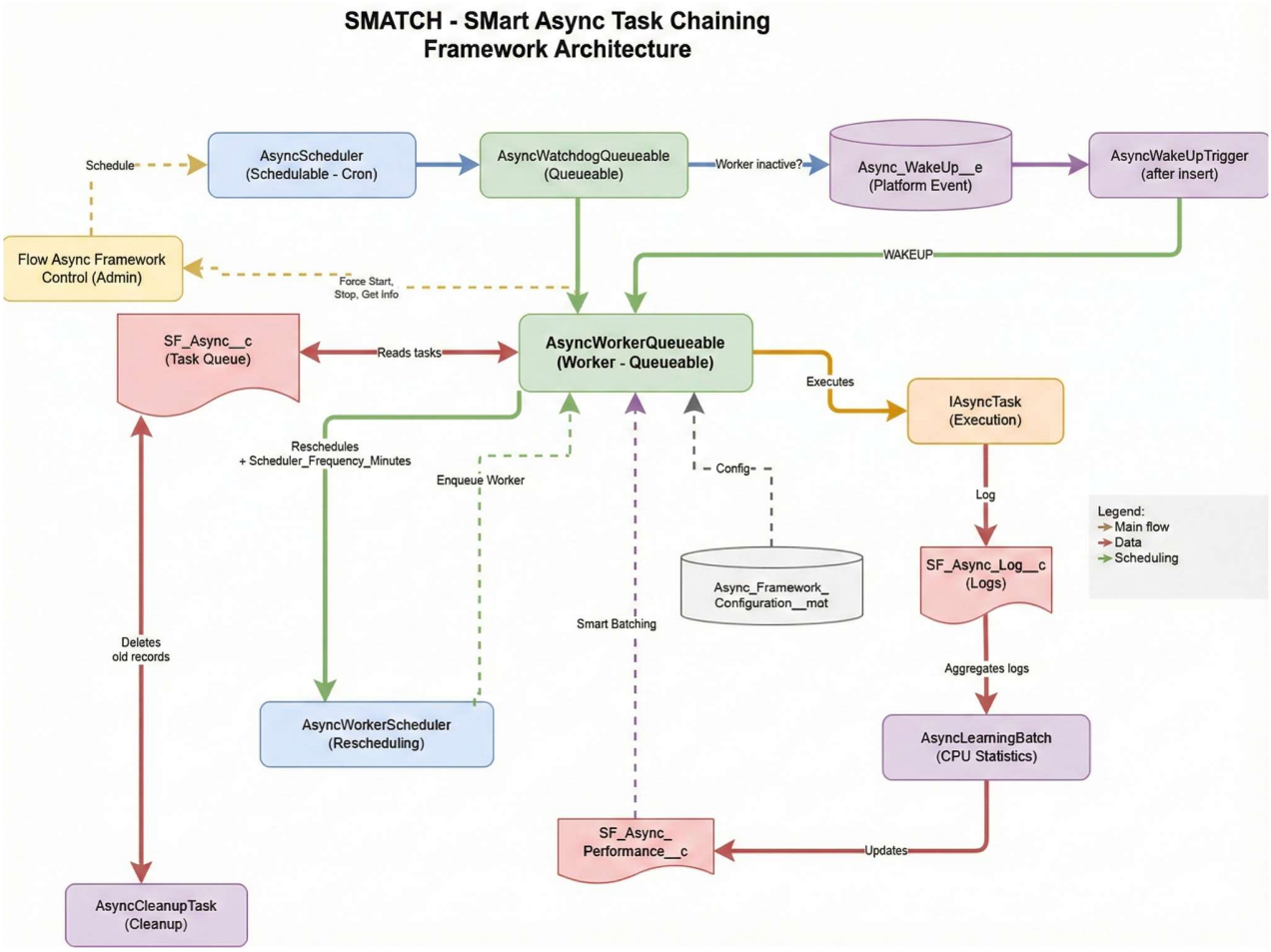
Benefit	Description
Reuse	Single framework for all async tasks in the project
Maintainability	Centralized configuration and evolution
Visibility	Backlog, performance, and error tracking
Reliability	Limits respected, error handling, automatic recovery
Scalability	Learning batch to dynamically adjust performance

I.F. Summary and Architecture

SMATCH aims to become the **standard foundation** for async execution on Salesforce: a single, configurable, and observable solution that reduces technical debt and improves background processing quality.

Main flow:

6. **AsyncScheduler** (cron) → executes **AsyncWatchdogQueueable**
7. **AsyncWatchdogQueueable** → checks if active Workers exist; if not, publishes Async_WakeUp__e (WAKEUP)
8. **AsyncWakeUpTrigger** (after insert on Async_WakeUp__e) → enqueues **AsyncWorkerQueueable**
9. **AsyncWorkerQueueable** → reads SF_Async__c, executes IAsyncTask tasks, logs, reschedules **AsyncWorkerScheduler**
10. **AsyncLearningBatch** → feeds SF_Async_Performance__c for Smart Batching



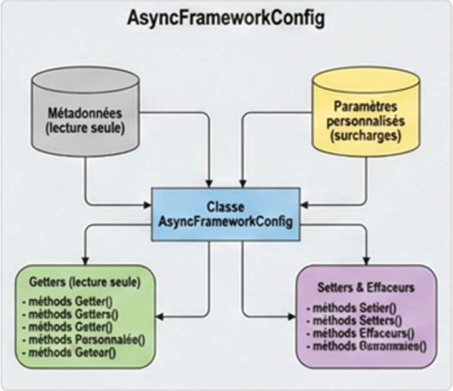
II. Implementation Choices

This chapter details technical decisions and the description of methods, triggers, and configurations. Each section starts with a justification of the choice, then describes the elements in detail.

II.1. Custom Metadata Configuration

Justification: Constants (CPU limit, batch sizes, scheduler frequency) must be modifiable without deployment. Custom Metadata with getInstance() does not consume SOQL queries.

AsyncFrameworkConfig



Utility class that reads configuration from Async_Framework_Configuration__mdt.Default_Configuration. Provides default values if metadata is not deployed.

Method	Inputs	Outputs	Description
`clearConfig()`	None	void	Clears cache after metadata update
`getSafeCpuLimit()`	-	Integer	Safe CPU limit in ms (default 45000)
`getMinBatchSize()`	-	Integer	Minimum batch size (default 1)
`getMaxBatchSize()`	-	Integer	Max size (reads `Async_Runtime_Config__c.Max_Batch_Size_Override__c` first)
`getMaxNumberWorker()`	-	Integer	Max number of workers (default 32)
`getParallelLowThreshold()`	-	Integer	Threshold for parallelization (default 200)
`getTasksPerWorkerDefault()`	-	Integer	Default max tasks per Worker (200)
`getDefaultCpuCostMs()`	-	Integer	Default CPU cost per task (1000)
`getCpuReserveMs()`	-	Integer	CPU reserve before limits (2000)
`getDmlSafeThreshold()`	-	Integer	DML threshold before batch reduction (75)
`getDaysToAnalyze()`	-	Integer	Days for statistics (7)
`getDaysToKeep()`	-	Integer	Days before old tasks deletion (30)
`getDefaultSchedulerFrequencyMinutes()`	-	Integer	Scheduler frequency (1)
`setMaxBatchSizeOverride(Integer)`	newSize	Void	Runtime override (Custom Settings)
`clearMaxBatchSizeOverride()`	-	Void	Restores metadata value

Async_Framework_Configuration__mdt (main fields)

Field	Type	Default	Description
Before_Cancelling__c	Number	3	Number of retries before cancelling a task
CPU_Reserve_ms__c	Number	2000	CPU safety margin (ms) reserved in the worker loop
Cleanup_Batch_Size__c	Number	2000	Batch size for cleanup deletion operations
Cleanup_CPU_Reserve_ms__c	Number	5000	CPU safety margin (ms) for cleanup task
DML_Safe_Threshold__c	Number	75	DML row threshold before worker stops to publish REDUCE_BATCH_SIZE (max 150)
Days_To_Analyze__c	Number	7	Number of days to analyze for historical performance
Days_To_Keep__c	Number	30	Number of days to keep records before cleanup
Delete_Pending_Max_Iterations__c	Number	10	Max iterations for delete pending operation
Default_CPU_Cost_ms__c	Number	1000	Default CPU cost (ms) when no performance data exists
Execution_Order__c	Number	99	Execution order for cleanup tasks
Max_Batch_Size__c	Number	200	Maximum tasks per batch
Max_Number_Worker__c	Number	32	Maximum number of parallel workers allowed
Min_Batch_Size__c	Number	1	Minimum tasks per batch
Parallel_Low_Threshold__c	Number	200	When effectiveLimit ≤ value, parallel workers are spawned
Safe_CPU_Limit__c	Number	45000	CPU safety margin: 45s of 60s available for worker
Safety_Margin_Multiplier__c	Number	1	Multiplier applied to average CPU time
Scheduler_Frequency_Minutes__c	Number	1	Scheduler run frequency in minutes
Status_Success__c	Text	Success	Status value for successful task execution
Tasks_Per_Worker_Default__c	Number	200	Default max tasks per worker when not set on SF_Async__c
Worker_number__c	Picklist	1	Number of workers to start

II.2. Platform Event and Trigger

Justification: The Async_WakeUp__e event allows waking the Worker in real time when a task is inserted or an external system requests it. The trigger enqueues the Worker without polling.

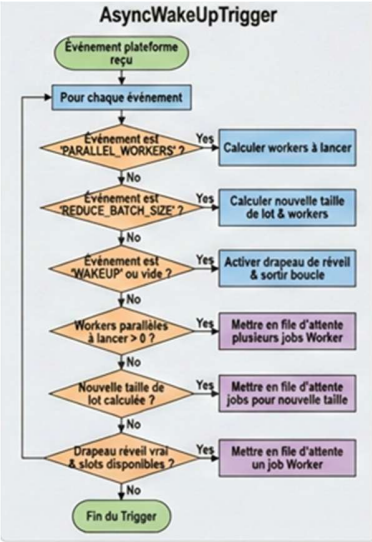
Async_WakeUp__e (Platform Event)

Field	Type	Description
Event_Type__c	Text	WAKEUP, REDUCE_BATCH_SIZE, PARALLEL_WORKERS, ERROR
Batch_Size__c	Number	Batch size (REDUCE_BATCH_SIZE, PARALLEL_WORKERS)
Workers_Count__c	Number	Worker count (REDUCE_BATCH_SIZE)
Error_Message__c	Text	Message or parseable integer (REDUCE_BATCH_SIZE)
Class_Name__c	Text	Affected class

Related_Record_Id__c	Text	SF_Async__c ID
Target_Record_Id__c	Text	Target

AsyncWakeUpTrigger

Context: after insert on Async_WakeUp__e.



Logic:

- **PARALLEL_WORKERS**: enqueues up to min(Batch_Size__c, 64, 50) Workers
- **REDUCE_BATCH_SIZE**: updates Async_Runtime_Config__c, enqueues additional Workers
- **WAKEUP** or empty: enqueues 1 Worker if Queueable slots are available

II.3. Worker and Smart Batching

Justification: The Worker must dynamically adapt batch size based on historical performance and platform limits. Smart Batching avoids errors and optimizes throughput.

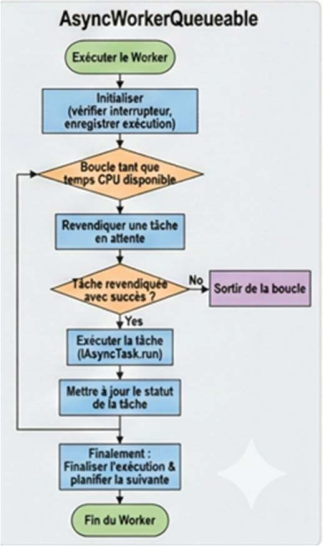
AsyncWorkerQueueable

Input: QueueableContext (Salesforce)

Behavior:

11. Selects next task class (SF_Async__c not executed, sorted by Scheduled_Time__c, Execution_Order__c)
12. Calculates effectiveLimit = min(CPU limit/avg cost, Tasks_Per_Worker_Limit, Max_Tasks_Per_Run_Learned)
13. Selects tasks with FOR UPDATE
14. Executes each task via IAsyncTask.run()

15. On callout/DML error: learns Max_Tasks_Per_Run_Learned__c = 1
16. On approaching DML limit: publishes REDUCE_BATCH_SIZE
17. Reschedules via AsyncWorkerScheduler or spawns parallel Workers



II.4. Scheduler and Watchdog

Justification: Two scheduling modes (multi-cron or single self-rescheduling) based on needs. The Watchdog ensures continuity even in case of failure.

AsyncScheduler

Schedulable executed by cron jobs. Enqueues **AsyncWatchdogQueueable**.

Method: execute(SchedulableContext) → enqueue Watchdog

AsyncSchedulerSelfRescheduling

Single job that reschedules itself after each execution. Used in "single" mode.

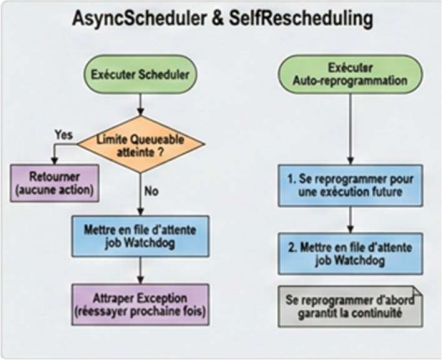
AsyncWatchdogQueueable

Checks if Workers are active (query AsyncApexJob). If not, publishes Async_WakeUp__e(Event_Type__c = 'WAKEUP').

AsyncWorkerScheduler

Reschedules the Worker with a frequency in minutes (from metadata or parameter).

Method	Inputs	Outputs	Description
`scheduleInMinutes(Integer)`	freq or null	String (jobId)	Schedules Worker, null = metadata



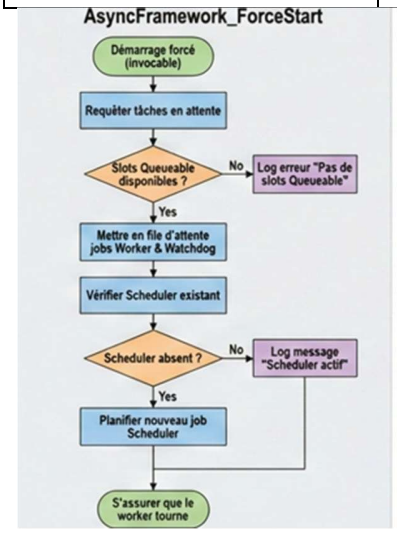
II.5. Flow Invocable Actions

Justification: Allow administrators to control the framework without code or deployment.

AsyncFramework_ForceStart

Starts the Worker and Watchdog. Schedules the Scheduler if absent.

Output	Type	Description
message	String	Task status, Worker, Watchdog, Scheduler



AsyncFramework_Diagnostic

Displays framework state: pending tasks, error logs, Workers, Schedulers, Statistics/Cleanup scheduled tasks.

Output	Type	Description
message	String	Complete text report

AsyncFramework_StopSchedulers

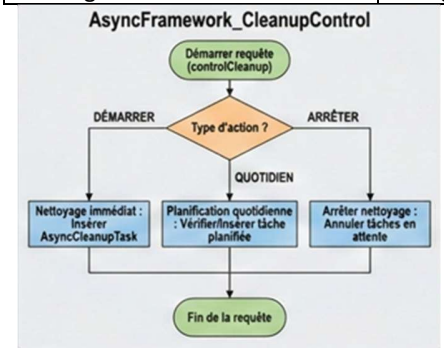
Stops all Async Schedulers (Async Framework Scheduler, AsyncWorker).

Output	Type	Description
message	String	List of stopped jobs or errors

AsyncFramework_CleanupControl

Controls data cleanup. Actions: START (immediate), START_DAILY (daily scheduling), STOP (cancellation).

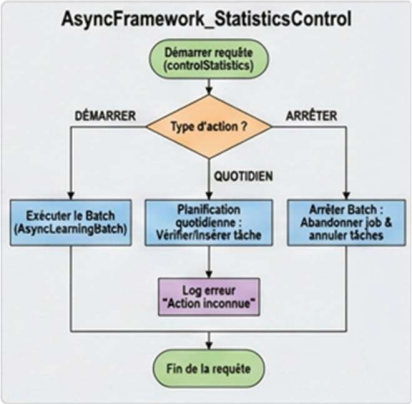
Input	Type	Description
action	String	START, START_DAILY, STOP
Output	**Type**	**Description**
message	String	Action result



AsyncFramework_StatisticsControl

Controls performance statistics. Actions: START, START_DAILY, STOP.

Input	Type	Description
action	String	START, START_DAILY, STOP
Output	**Type**	**Description**
message	String	Result

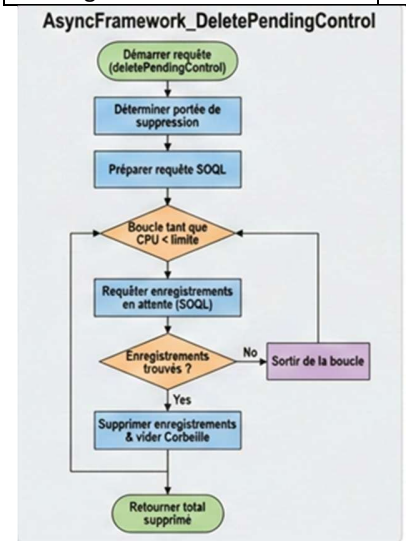


AsyncFramework_DeletePendingControl / AsyncFramework_DeletePendingFlow

Deletes pending tasks. Option: delete all or preserve AsyncCleanupTask and AsyncStatisticsTask.

Input (Control)	Type	Description
deletePending	Boolean	true = all, false = preserve

		stats/cleanup
Input (Flow)	**Type**	**Description**
deleteAllCollection	List<List<Boolean>>	[[true]] or [[false]]
Output	**Type**	**Description**
recordsDeleted	Integer	Number deleted
message	String	Result message



AsyncSchedulerSetup

Scheduler planning management from Flow. Actions: GET_INFO, SCHEDULE_EVERY_MINUTE, SCHEDULE_CUSTOM, UPDATE_FREQUENCY_METADATA.

Input	Type	Description
action	String	GET_INFO, SCHEDULE_EVERY_MINUTE, SCHEDULE_CUSTOM, UPDATE_FREQUENCY_METADATA
frequencyMinutes	Integer	For SCHEDULE_CUSTOM, UPDATE_FREQUENCY_METADATA
jobName	String	Job name
useSingleScheduler	Boolean	true = AsyncSchedulerSelfRescheduling

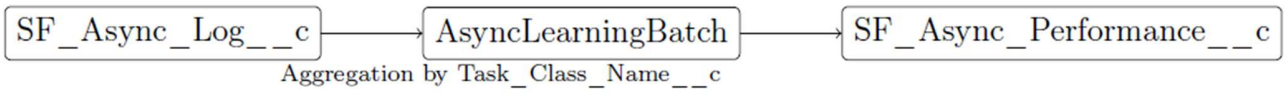
II.6. Built-in Tasks (IAsyncTask)

AsyncCleanupTask

Deletes SF_Async__c and SF_Async_Log__c executed or cancelled more than Days_To_Keep__c days ago. Configurable via Cleanup_Batch_Size__c and Cleanup_CPU_Reserve_Ms__c.

AsyncStatisticsTask

Launches the **AsyncLearningBatch** which aggregates SF_Async_Log__c logs and updates SF_Async_Performance__c (Average_CPU__c, Sample_Size__c).



II.7. Runtime and Fallback

Async_Runtime_Config__c (Custom Settings)

Field	Description
Max_Batch_Size_Override__c	Temporary max batch override
Active_Worker_Count__c	Number of active workers (used by REDUCE_BATCH_SIZE)

AsyncReduceBatchSizeJob

Queueable fallback when EventBus.publish fails (e.g. DML limit). Updates Custom Settings and can republish the event.

III. Administrator Manual

This section describes how an administrator uses the SMATCH application.

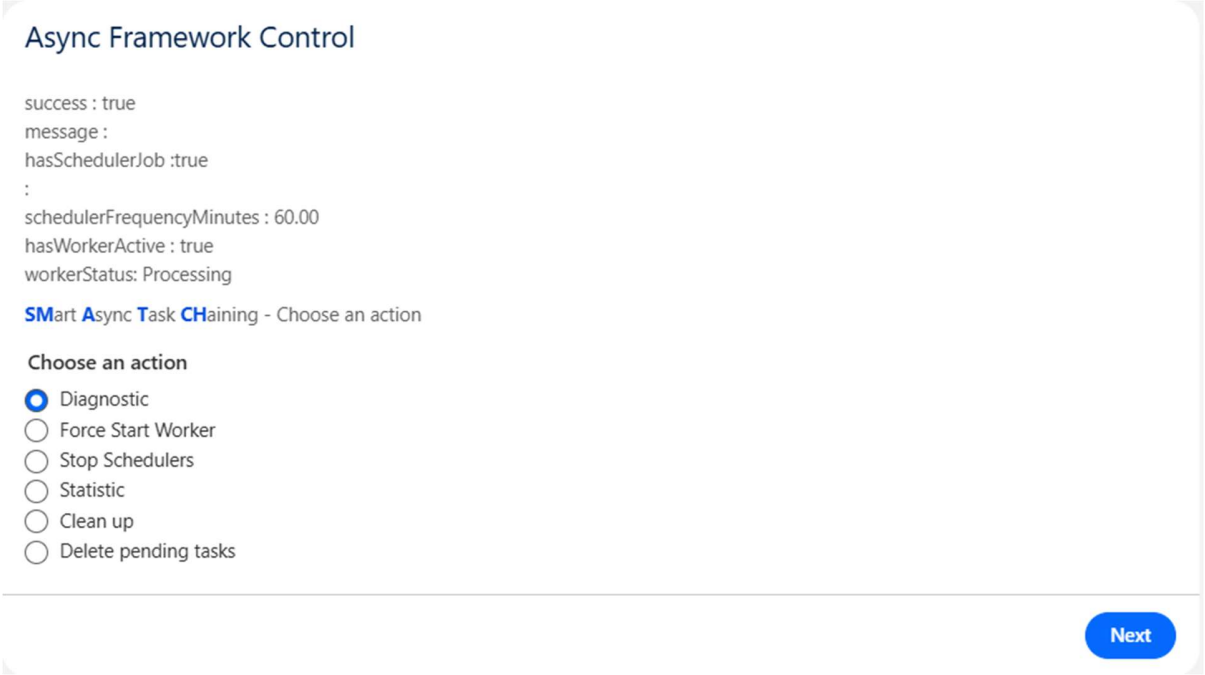
III.1. Accessing the Smatch Application

18. **App Launcher** → search for **Smatch**
19. The Smatch application displays:

- The **Async_Framework_Control** Flow (main menu)
- Optionally a **Dashboard** of tasks

III.2. Flow Main Menu

On Flow startup, a menu offers the following actions:



Choice	Description
Diagnostic	Displays complete framework state
Force Start Worker	Restarts Worker and Watchdog
Stop Schedulers	Stops all Async schedulers
Statistic	Statistics submenu
Clean up	Cleanup submenu
Delete Pending Tasks	Delete pending tasks

Suggested screenshot: main menu screen (flowDashboard.png if available)

III.3. Diagnostic

Async Framework Control

```
=== ASYNC FRAMEWORK DIAGNOSTIC ===
Date/time: 13/02/2026 12:02

PENDING TASKS: 0

RECENT TASKS:
- Daily statistics task | AsyncStatisticsTask | Executed: false | Cancelled: false
- Daily statistics task | AsyncStatisticsTask | Executed: true | Cancelled: false
- Daily Cleanup | AsyncCleanupTask | Executed: false | Cancelled: false
- Daily Cleanup | AsyncCleanupTask | Executed: true | Cancelled: false
- Daily statistics task | AsyncStatisticsTask | Executed: false | Cancelled: true

ERRORS (2h): 0

WORKERS (2h): 5
- Completed (13/02/2026 12:02)
- Completed (13/02/2026 12:01)
- Completed (13/02/2026 12:00)
- Completed (13/02/2026 11:59)
- Completed (13/02/2026 11:58)

SCHEDULERS: 1
- AsyncWorker 20260213120300 | State: WAITING | Next: 13/02/2026 12:03

STATISTICS SCHEDULED: Yes
Frequency: Daily | Next calculation: 13/02/2026 20:06

CLEANUP SCHEDULED: Yes
Frequency: Daily | Next execution: 13/02/2026 19:41

WATCHDOG SCHEDULED: No

WORKER SCHEDULED: Yes
Name: AsyncWorker 20260213120300 | Frequency: 1 min | Next: 13/02/2026 12:03

=== END OF DIAGNOSTIC ===
```

20. Select **Diagnostic**
21. The Flow displays a report containing:

- Number of pending tasks
- Recent tasks (executed, cancelled)
- Error logs from last 2 hours
- Recent Workers
- Active Schedulers
- Scheduled Statistics and Cleanup tasks
- Watchdog and Worker state

Suggested capture: diagnostic result (flowdiagnostic.png if available)

III.4. Force Start Worker

Use this action to:

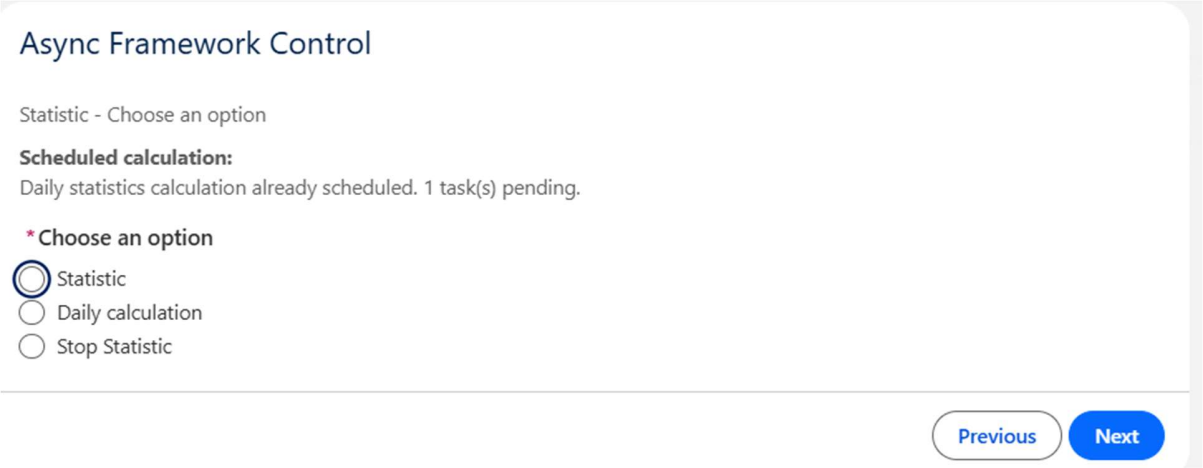
- Restart the system after a stop
- Process pending tasks immediately

The Flow enqueues the Worker and Watchdog, and schedules the Scheduler if absent.

III.5. Stop Schedulers

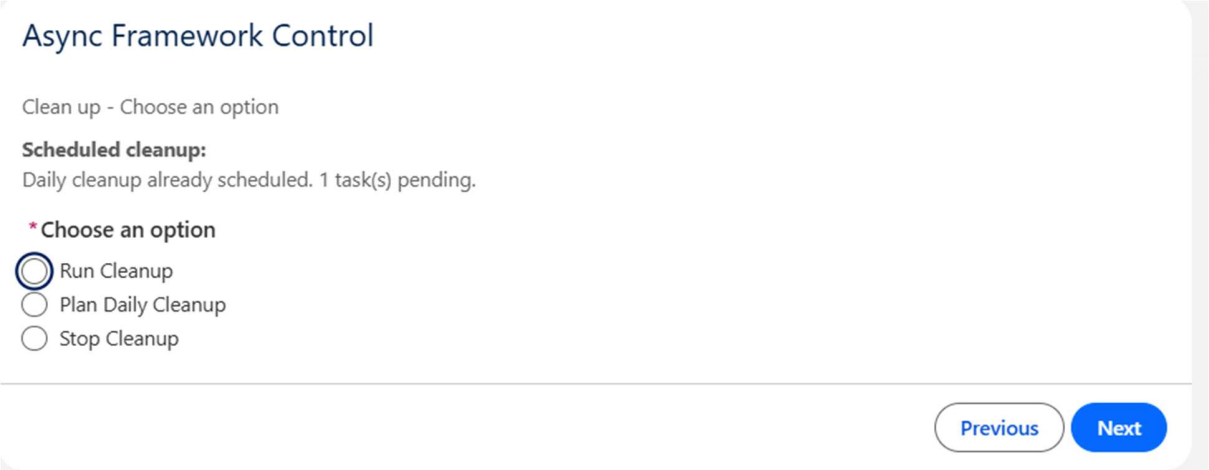
Stops all framework cron jobs (Async Framework Scheduler, AsyncWorker). Useful for maintenance or troubleshooting.

III.6. Statistic Submenu



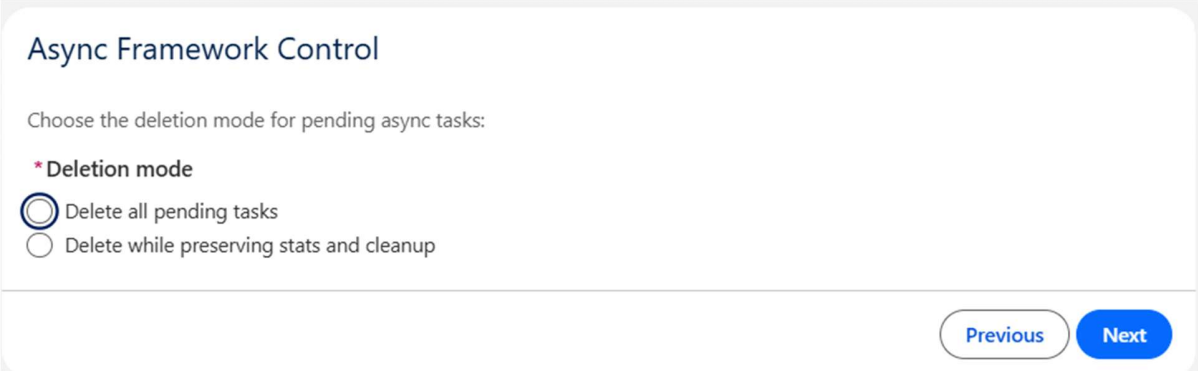
Action	Description
Start Statistic	Immediately launches a statistics calculation
Plan Daily Calculation	Schedules a daily calculation
Stop Statistic	Cancels in-progress statistics tasks

III.7. Clean up Submenu



Action	Description
Run Cleanup	Immediate cleanup of old data
Plan Daily Cleanup	Schedules daily cleanup
Stop Cleanup	Cancels scheduled cleanup tasks

III.8. Delete Pending Tasks



22. Select **Delete Pending Tasks**
23. Choose mode:

- **Delete All**: deletes all pending tasks
- **Preserve Stats**: preserves AsyncCleanupTask and AsyncStatisticsTask

24. Confirm deletion

III.10. Advanced Configuration (Setup)

- **Custom Metadata**: Setup → Custom Metadata Types → Async Framework Configuration → Manage Records → Default Configuration

- **Custom Settings**: Setup → Custom Settings → Async Runtime Config → Manage
- **Remote Site Settings**: for tasks with callout (e.g. Calculator)

IV. Developer Manual

This chapter explains how to use the SMATCH framework to create asynchronous tasks. The **CalculatorCalloutTask** example and a conceptual **CreateAccountTask** example illustrate best practices.

IV.1. IAsyncTask Interface

Any task executed by SMATCH must implement IAsyncTask:

```
public interface IAsyncTask {
    void run(SF_Async__c contextRecord);
}
```

Input: contextRecord – SF_Async__c record containing:

- Target_Record_Id__c: target record ID
- Additional_Information__c: additional JSON or text data
- Task_Class_Name__c: name of class to execute

IV.2. CreateAccountTask Example (Conceptual)

Simple task that creates an Account. Not in the package but illustrates the minimal case.

```
public class CreateAccountTask implements IAsyncTask {
    public void run(SF_Async__c contextRecord) {
        Account acc = new Account(
            Name = 'Account created at ' + System.now().format('yyyy-MM-dd
HH:mm')
        );
        insert acc;
    }
}
```

Creating a task:

```
SF_Async__c task = new SF_Async__c(
    label__c = 'Create Account',
    Task_Class_Name__c = 'CreateAccountTask',
    Scheduled_Time__c = System.now(),
    Execution_Order__c = 1,
    Before_Cancelling__c = 3
);
```

```
insert task;
EventBus.publish(new Async_WakeUp__e(Event_Type__c = 'WAKEUP'));
```

IV.3. CalculatorCalloutTask Example

Prerequisites

- Calculator_Result__c object (Operand_A__c, Operand_B__c, Result__c)
- Custom Label Calculator_Endpoint (base URL, e.g. https://newton.vercel.app)
- Remote Site Settings to allow callout

Code

```
public class CalculatorCalloutTask implements IAsyncTask {
    public void run(SF_Async__c contextRecord) {
        Id targetId = contextRecord.Target_Record_Id__c;
        if (targetId == null) {
            throw new AsyncException('Target_Record_Id__c is required.');
        }
        Calculator_Result__c rec = [SELECT Id, Operand_A__c, Operand_B__c
            FROM Calculator_Result__c WHERE Id = :targetId LIMIT 1];
        Integer a = rec.Operand_A__c != null ? Integer.valueOf(rec.Operand_A__c)
: 0;
        Integer b = rec.Operand_B__c != null ? Integer.valueOf(rec.Operand_B__c)
: 0;
        String baseUrl = System.Label.Calculator_Endpoint;
        if (String.isBlank(baseUrl)) baseUrl = 'https://newton.vercel.app';
        String endpoint = baseUrl + '/api/v2/simplify/' +
EncodingUtil.urlEncode(a+'+'+b, 'UTF-8');
        HttpRequest req = new HttpRequest();
        req.setEndpoint(endpoint);
        req.setMethod('GET');
        req.setTimeout(120000);
        HttpResponse res = new Http().send(req);
        if (res.getStatusCode() != 200) {
            throw new AsyncException('API error: ' + res.getStatusCode());
        }
        Map<String, Object> parsed = (Map<String, Object>)
JSON.deserializeUntyped(res.getBody());
        rec.Result__c = Decimal.valueOf(String.valueOf(parsed.get('result')));
        update rec;
    }
}
```

Execution Flow

25. Create a Calculator_Result__c record (Operand A, Operand B)
26. Create an SF_Async__c task with Task_Class_Name__c = 'CalculatorCalloutTask' and Target_Record_Id__c = Calculator Result ID
27. Publish Async_WakeUp__e(Event_Type__c = 'WAKEUP') or wait for the Scheduler
28. The Worker executes CalculatorCalloutTask.run() which calls the API and updates Result__c

Detailed configuration: see `docs/CALCULATOR_SETUP_FR.md` (French)

IV.4. SF_Async__c Fields Used by the Developer

Field	Required	Description
Task_Class_Name__c	Yes	Apex class name (e.g. CalculatorCalloutTask)
Target_Record_Id__c	Per task	Target record ID
Scheduled_Time__c	Yes	Execution datetime (past = immediate)
Execution_Order__c	No	Relative order (default 99)
Before_Cancelling__c	No	Max attempts before cancellation (default 3)
Tasks_Per_Worker_Limit__c	No	Task limit per Worker for this task
label__c	No	Display label
Additional_Information__c	No	Additional data (JSON, etc.)

IV.5. Anti-Patterns to Avoid

Practice	Problem	Solution
DML then Callout in same transaction	"Uncommitted work" error	Do callout before any DML, or in separate transaction
Task too long	CPU timeout	Split into multiple tasks, reduce batch size
Non-existent class in Task_Class_Name__c	TypeException at runtime	Verify class name (with namespace if packaged)
No error handling	Silent failure	Throw explicit exceptions, check SF_Async_Log__c
Target_Record_Id__c null for task that needs it	NullPointerException	Validate inputs at start of run()
Modify configuration without clearConfig	Stale values	Call AsyncFrameworkConfig.clearConfig() after metadata update

IV.6. Best Practices

- Use AsyncFrameworkConfig.getBeforeCancelling() and getExecutionOrder() for default values
- Test with tasks scheduled in the past for immediate execution
- Check SF_Async_Log__c and SF_Async_Performance__c for diagnostics
- For callouts: configure Remote Site Settings and Custom Labels for URLs

---

V. Appendix

V.A. Package Files

Apex Classes (excerpt)

Class	Role
AggregateResultIterable	Utility – Iterator for AggregateResult
AggregateResultIterator	Utility – AggregateResult iterator
AsyncCleanupTask	Task – Logs and statistics cleanup
AsyncFrameworkConfig	Framework – Metadata configuration
AsyncFramework_CleanupControl	Flow – Cleanup control
AsyncFramework_DeletePendingControl	Flow – Delete pending tasks
AsyncFramework_DeletePendingFlow	Flow – Delete pending (Flow)
AsyncFramework_Diagnostic	Flow – Framework diagnostic
AsyncFramework_ForceStart	Flow – Force worker start
AsyncFramework_StatisticsControl	Flow – Statistics control
AsyncFramework_StopSchedulers	Flow – Stop schedulers
AsyncLearningBatch	Task – Performance learning
AsyncReduceBatchSizeJob	Batch – Batch size reduction
AsyncScheduler	Scheduler – Main scheduler
AsyncSchedulerSelfRescheduling	Scheduler – Self-rescheduling
AsyncSchedulerSetup	Flow – Scheduler setup
AsyncStatisticsTask	Task – Statistics calculation
AsyncTaskScheduleInfo	Utility – Schedule info
AsyncWatchdogQueueable	Watchdog – Worker monitoring
AsyncWorkerQueueable	Worker – Asynchronous task execution
AsyncWorkerScheduler	Scheduler – Worker scheduling

Objects and Metadata

Element	Description
SF_Async__c	Async tasks
SF_Async_Log__c	Execution logs
SF_Async_Performance__c	Statistics per class
Async_WakeUp__e	Platform Event
Async_Runtime_Config__c	Custom Settings
Async_Framework_Configuration__mdt	Custom Metadata
Calculator_Result__c	Example object

Triggers

Trigger	Context
AsyncWakeUpTrigger	after insert Async_WakeUp__e

Flows

Flow	Description
Async_Framework_Control	Main control interface

V.B. Test Coverage

The classes and trigger have unit tests. Coverage summary (typical state):

Class	Coverage	Covered	Total
AsyncWorkerQueueable	89%	196	221
AsyncFramework_ForceStart	90%	52	58
AsyncFramework_Diagnostic	90%	80	89
AsyncSchedulerSetup	90%	134	149
AsyncFramework_StatisticsControl	91%	43	47
AsyncScheduler	91%	10	11
AsyncFramework_StopSchedulers	91%	29	32
AsyncWorkerScheduler	93%	26	28
AsyncLearningBatch	94%	32	34
AsyncReduceBatchSizeJob	94%	17	18
AsyncStatisticsTask	95%	18	19
AsyncSchedulerSelfRescheduling	95%	21	22
AsyncFramework_DeletePendingControl	95%	56	59
AsyncFramework_CleanupControl	96%	44	46
AsyncCleanupTask	97%	31	32
AsyncFrameworkConfig	98%	93	95
AsyncWatchdogQueueable	100%	28	28
DateTimeFormatUtil	100%	5	5
AggregateResultIterator	100%	11	11
AggregateResultIterable	100%	2	2
AsyncWakeUpTrigger	100%	46	46
AsyncFramework_DeletePendingFlow	100%	8	8
Total	**93%**	**982**	**1060**